

Transformer Fleet Health & Replacement Planning

Which transformers to replace or inspect within the budget, and why

Team Rayquaza · HCLTech Future Ready AI Engineer Hackathon · Data Science & ML edition · 11 Oct 2026

1 The problem

A utility can afford to fix only a few transformers a year

- ~400 power transformers. A failure can cut power to a hospital or a village and costs far more than a planned replacement.
- Oil tests (gases dissolved in the oil) are the best health signal, but many units have never been tested.
- Failures are rare, so a model that says "nothing fails" looks accurate and is useless.

400 transformers in the fleet

38% never oil-tested

4.3% of snapshots fail within 12 months

User and decision

The asset-planning engineer. With this year's budget: which transformers to replace, which to inspect, which to leave, backed by probabilities they can trust and a reason for every choice.

2 Is this a real problem? Yes: industry relevance

Every grid operator faces this decision every year

1 Ageing fleets, fixed budgets

Much of the installed transformer base is decades old, and utilities can replace only a small share each year. Choosing which ones is the core asset-management decision.

2 Failures are expensive and slow to fix

An unplanned failure means emergency replacement, outage penalties and lost supply. New large units can take many months to arrive, so replacements must be planned ahead.

3 Who loses power matters

A hospital, an industrial site and a village feeder are not equal. Planning has to weigh consequence, not just condition.

4 Condition data is sparse

Oil tests (dissolved-gas analysis, the industry-standard health check: IEEE C57.104 / IEC 60599) are sampled, not continuous, and many units have never been tested.

Our formula is the industry's formula

risk = probability of failure × consequence of failure

Risk-based asset management (ISO 55000) and regulator methods such as the UK's network asset risk methodology rank assets the same way. We add calibrated probabilities, honest uncertainty and a budget optimiser.

Who has this problem

Distribution and transmission utilities, industrial plants, data centres, railways, renewable-energy substations, large campuses and hospitals.

3

What the system delivers

Every number comes from code; every recommendation has a reason

Failure probability + range

"TR-0102: 19% chance of failing in 12 months (likely 12–28%)". Calibrated, wider when we know less.

Health score per field

100 minus points for gas condition (IEEE C57.104 levels), gas trend, peak load and age. Blank, never "healthy", without an oil test.

Risk ranking

Risk = probability × impact (hospital 5, industry 3, urban 2, rural 1, × customers served), with top factors per unit.

Budget plan

Knapsack picks replace / inspect / nothing within budget, compared with "replace the oldest". Every action needs approval.

Honest about missing data

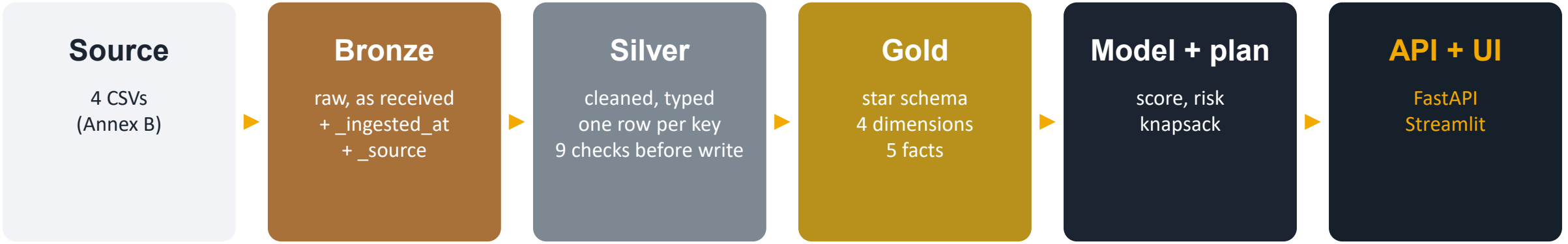
Untested units: scored_low_confidence, wide range, inspect before any replacement.

New data → new plan

Load a CSV (e.g. inspection results): validate → rebuild → re-score → re-plan → before/after comparison.

4 Architecture: the medallion pipeline

Raw → cleaned → business-ready · one command rebuilds everything in ~3 s: python -m src.pipeline



Bronze = the replay tape

Every file kept exactly as received, including what /ingest saved. A bug later is fixed and re-run without the source. Audit trail of what arrived when.

Silver = one place for quality

IDs tidied, duplicates removed, ppb→ppm fixed, gaps left NULL. 9 HCL checks run BEFORE anything is written: bad data stops the run.

Gold = shaped for reading

Star schema with shared dimensions. fact_snapshot is the single, leakage-safe input for the model AND the API: one function builds features for both.

5

Star schema: 4 shared dimensions, 5 fact tables

Conformed dimensions: every fact can be sliced the same way

Fact table ↓ / Dimension →	dim_transformer	dim_date	dim_run	dim_action	Grain: one row =
fact_oil_test	✓	✓			one transformer × one oil test
fact_operation	✓	✓			one transformer × one month
fact_snapshot (model input)	✓	✓			one transformer × one quarterly snapshot
fact_risk	✓	✓	✓		one scoring run × one transformer
fact_plan	✓	✓	✓	✓	one plan × one chosen action

Star, not snowflake

Customer-type weight lives inside dim_transformer: one join fewer, a little repetition accepted.

Surrogate keys

transformer_key is our own stable integer: business IDs arrive messy ("tr-0045 ").

Run history kept

dim_run, fact_risk and fact_plan are never wiped by a rebuild, so before/after new data can be compared.

6

Every gold table and how it is used

Built by src/pipeline.py and src/features.py · run history written by the API and the UI

Table	Rows	Key columns / measures	Used for
dim_transformer	400	transformer_key, customer_type, customers, region, rating_class, type_weight, impact_weight	risk = p × impact · pricing by size · UI filters
dim_date	1,188	date_key (20250101), year, quarter, month, is_snapshot_date	slice any fact by month / quarter
dim_action	6	action_key, action (replace/inspect), rating_class, cost	plan prices · fact_plan
dim_run	per run	run_id, created_at, kind, as_of, budget, model_version	compare runs before / after new data
fact_oil_test	1,016	6 gases (ppm), total combustible gas, unit_fixed	gas history · analysis
fact_operation	14,112	load %, top-oil temperature, age, is_failure, is_repair	stress history · failure counts
fact_snapshot	4,800	21 features + failed_next_12m (3,200 labelled, 138 positive)	the model's training table
fact_risk	per run	probability, 90% range, health, impact, risk	API /predict · UI re-score
fact_plan	per plan	action, cost, risk before, risk removed	API /plan · UI Approve

7

The grain, and how we stop the model seeing the future

One row = one transformer at one quarterly snapshot date

History before the date → the INPUTS (21 features)

Snap-shot

Next 12 months → the ANSWER

SQL window functions

ROW_NUMBER() → the newest usable oil test before the snapshot.
LAG() → the previous test, so we know how fast gases are rising.
A repair resets the pattern: tests count from the month after it.

Labels are NULL, not 0

Quarters after 2025-10-01 don't have 12 months of future in the data yet. Treating "not seen yet" as "did not fail" would quietly corrupt training.

Leakage test, proven

150 random rows recomputed in plain pandas from data before the snapshot only: they match. We planted a leak on purpose (tests 3 months in the future): the test failed. Reverted → passes.

8 How the model is trained

src/model.py · scikit-learn only · seed 42 everywhere

1 Split

Held-out = last 2 labelled quarters × 20% of transformers (160 rows, 9 failures), used ONCE.

2 Candidates

Baselines: age only, health-index rule. Models: LR + median, LR + missing flags, boosting with native NaN.

3 Validate

Grouped by transformer AND time-aware (train on earlier quarters), 5 repeats → mean and range.

4 Choose

Simplest within the noise of the best → logistic regression + median fill.

5 Calibrate

Platt (sigmoid): Brier 0.18 → 0.053. Isotonic rejected: overfits ~40 failures.

6 Range

30 bootstrap copies → 90% range. Untested units get a wider range (0.12 vs 0.07).

7 Explain

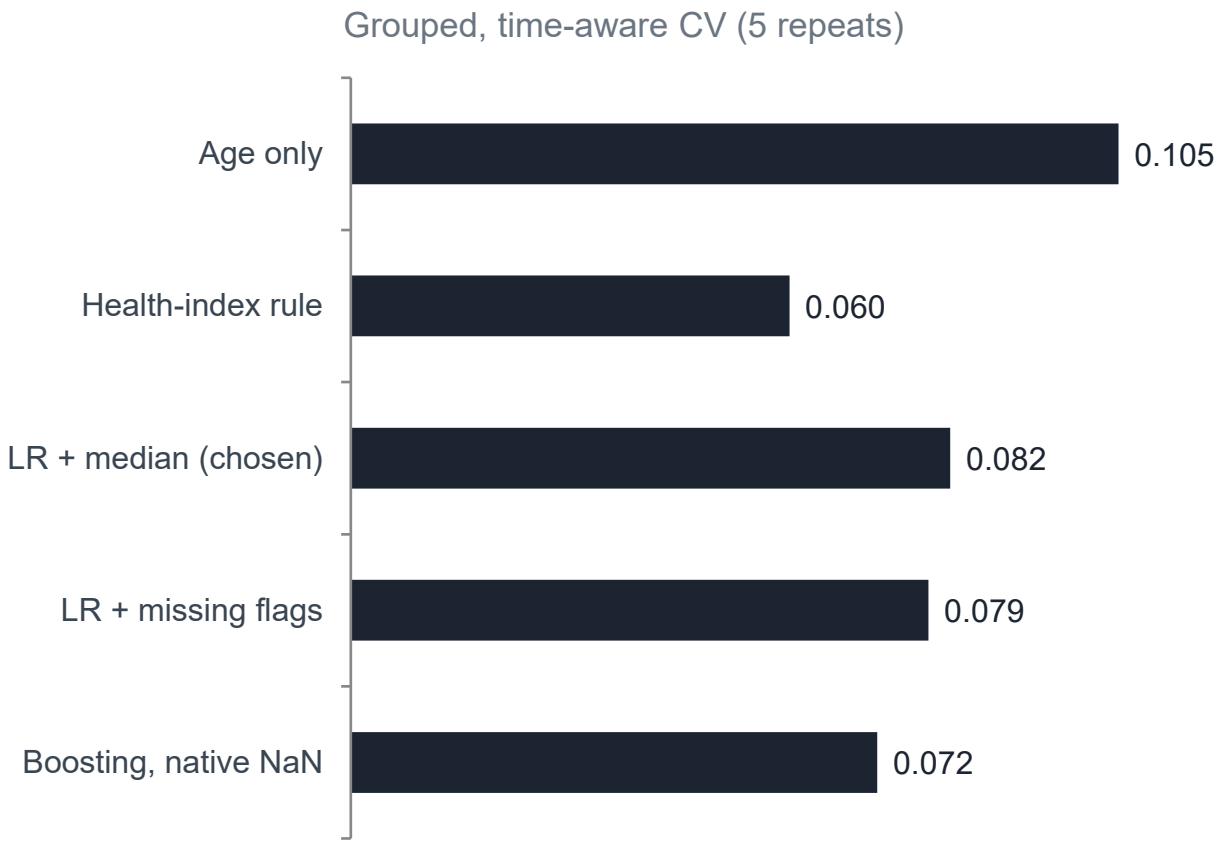
Top 3 drivers per unit; permutation importance globally (age leads).

8 Save + serve

models/model.joblib + model card; score() → probability, range, status, health, drivers.

9 Results, stated honestly

PR-AUC: are real failures near the top of the list? (guessing = 0.043)



Held-out, used once (9 failures): model 0.104 · age only 0.152 · health rule 0.356. All intervals overlap.

Calibration works

Brier score 0.182 → 0.053 after Platt scaling: a predicted 20% now means about 20%, which the risk formula needs.

Honest verdict

On our synthetic data the model does NOT beat age alone: failures there are driven mostly by age and the gas signal is faint. We keep it because an inspection result must be able to change a unit's risk, and age-only would ignore it. A 4-input version (age + gas/trend/load points) scored 0.117 and beat age-only: our next step.

10

From prediction to decision

Risk = calibrated probability × impact, then a knapsack within the budget

Hospital unit

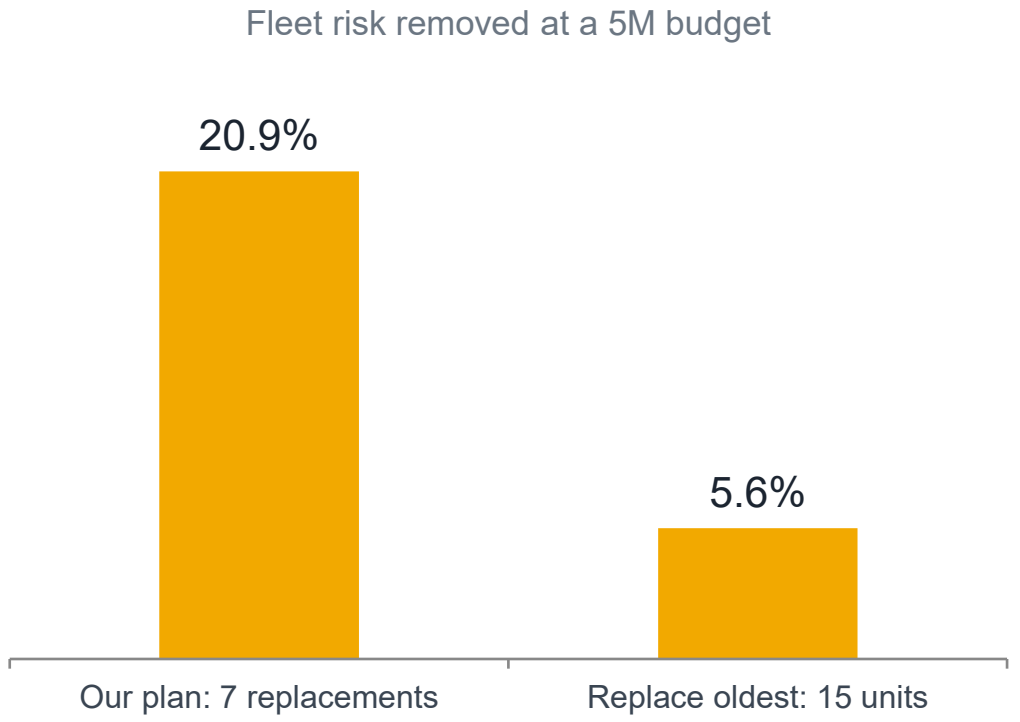
12,000 customers
impact 5 × 12,000 = 60,000
19% → risk 11,400

Rural unit

2,000 customers
impact 1 × 2,000 = 2,000
25% → risk 500

The hospital unit comes first, even though the rural one is more likely to fail.

- **Replace removes risk: (p − 1%) × impact**
- Inspect reveals hidden risk: (p_high − p) × impact: most valuable where we know least
- Never replace blind: no recent oil test → inspect only
- Knapsack (dynamic programming): ≤ 1 action per unit, cost ≤ budget
- Every plan is a proposal: an engineer approves it



Plus 226 inspections covering 377,943 of hidden risk, shown separately, never added. Held-out budget test: 10.0% vs 2.3%; acted on 4 real failures vs 2.

11

Mentor feedback, and what we built

"If an inspection shows it is worse than predicted, its risk should go up."



Live in the UI: the rehearsal file re-scores 40 transformers. A worse unit's risk rises, its range narrows, and the next plan replaces it.

Panel said (12:00 review)	What we did
Inspection results should raise risk	Inspection = new oil test → loader → rebuild → re-score → re-plan; before/after table in the UI
Assume replacement / inspection costs and the budget	replace 150k / 350k / 700k · inspect 10k / 15k / 25k · budget 5M (late drop 4M)
Which libraries for data, models, everything?	pandas/numpy · SQLite/SQLAlchemy · scikit-learn/joblib · FastAPI · Streamlit · pytest
Docker is optional	Removed; requirements.txt pinned to the exact versions we tested with

12

Every edge case, handled on purpose

All six HCL traps were planted in our data, so each one can be shown

Edge case	How we handle it	Where
No oil test at all (38%)	has_oil_test feature · low confidence · wider range · health blank, never "healthy" · inspect first	features · model · plan
Old units with few tests	low confidence (one test) · wide range	model
Look-alike pair: hospital vs rural	same probability, very different impact → the hospital ranks far higher	dim_transformer · plan
Gas jump from a unit change (ppb)	a test ≥ 200× the unit's other tests → ÷1000 and flagged (found exactly the 3 planted)	silver
Recent repair resets the pattern	gas features use only tests from the month after the latest repair / failure	features
Budget can't cover all high-risk units	knapsack takes the best risk per unit of money; the rest wait	plan
Duplicates, messy IDs, gaps	strip().upper() · one row per key, last loaded wins · gaps stay NULL	silver
Judges' files without our extra columns	no rating_class → medium (flagged) · no event → none	silver
Renamed column · bad rows in uploads	COLUMN_ALIASES config · rows rejected with a reason · rollback if validation fails	loader

13

The interface: the engineer's workflow

streamlit run ui/app.py · calls the same functions as the API

1

Fleet risk ranking

Where is the risk? KPIs, ranked table, filters, low-confidence badges.

2

Transformer detail

Why? Probability + range, health points per field, gas history with ppb fix and repairs.

3

Budget plan

What do we do? Our plan vs replace-oldest, removed vs revealed risk, Approve.

4

New data & what changed

Upload inspection results → re-score → before/after → plan changes.

5

Model & data quality

Reliability chart: does a predicted 20% mean ~20%? Plus the 9/9 validation report.

Where is the risk → why → what do we do → what changed → can we trust it

14

How we validated, including the AI's work

AI assistants wrote most of the code; we specified the design and checked everything

9/9

HCL validation checks

101

automated tests pass

36/36

independent data-spec checks

What we checked

Leakage test proven by planting a leak · row counts at every join (validate="many_to_one") · two runs give identical tables · every UI screen run headless against the real model · every reported number read for plausibility.

Mistakes we caught

1. Plan claimed "108.9% of risk removed": revealed risk was added to removed risk. Now reported separately.

2. A test passed for the wrong reason (missing column, not the future date). Fixed the test.

3. Scoring as of 2026-10-01 would have ignored the late-drop oil tests (dated Oct 1–10).

4. requirements pinned pandas 2.2.3; the code was tested on 3.0.0. Re-pinned.

5. A push added a nested repo pointer instead of the data. Removed.

15

Libraries, by job

Small, standard stack: nothing to break on venue wifi

Job	Library	Used for
Data generation	numpy, pandas	seeded synthetic fleet with planted edge cases, data card
Storage + pipeline	SQLite, SQLAlchemy, pandas	bronze / silver / gold, transactions, window-function SQL
Model	scikit-learn, joblib	logistic regression, boosting, imputation, pipelines, grouped CV, calibration, metrics
Budget optimiser	plain Python (dynamic programming)	knapsack: explainable line by line, no solver needed
Serving	FastAPI, Uvicorn, Pydantic v2, python-multipart	/predict /plan /ingest /model/info /explain /health
Interface + charts	Streamlit, matplotlib	5 screens, gas history, reliability chart
Testing	pytest, httpx	101 tests, including leakage and API contract

16

How it scales

The grain and the shared feature function stay; only the engine underneath changes

Area	Today	At scale
Storage	SQLite, one file	PostgreSQL; bronze as Parquet partitioned by month; indexes on (transformer_id, test_date) and (transformer_id, month)
Loading	full rebuild, ~3 s	incremental: watermark on _ingested_at, upsert into silver, only the new quarter's snapshots
History	SCD Type 1 (overwrite)	SCD Type 2 on dim_transformer: keep customer / size changes over time
Model	retrained by hand	scheduled monthly retraining, drift + calibration monitoring; versions already in dim_run
Optimiser	DP: 400 units × 1,000 budget steps	integer programming (OR-Tools / PuLP) or greedy by value-per-cost for 10k+ units
API	one Uvicorn process	stateless → several workers behind Nginx; nightly precomputed fact_risk
UI	scores live	reads precomputed fact_risk; live scoring only for what-if questions
Deployment	our laptops	containers on a Linux server

17

From prototype to product: taking it to market

Start with one utility, prove it on their own history, then grow by assets and by customers

1 Pilot

One utility region. Load their history, back-test: would our plan have caught last year's failures better than their current method?

2 Integrate

Connect real sources: lab oil-test results, meter / SCADA loading, GIS map of who each transformer feeds, work-order history.

3 Expand assets

Same pattern (grain + features + probability × consequence + budget optimiser) for breakers, cables, poles, substations.

4 Platform

Many utilities on one product: benchmarking across fleets, live sensor feeds, a model per utility, approvals built in.

Who pays, and how

Utilities and large asset owners. Subscription priced per asset monitored; deployed in their environment because grid data is sensitive.

The value we sell

More risk removed for the same budget (in our test: 20.9% vs 5.6% for replacing the oldest), fewer unplanned outages, and a defensible, auditable plan.

Why it can scale

Schema mapping is configuration, not code; one feature function for training and serving; every plan approved by an engineer, which builds trust.

18

With more time, we would…

Ordered by impact

1

Ship the 4-input model

Age + gas/trend/load points beat age-only in CV (0.117 vs 0.105). Re-run the held-out test once.

2

Real failure history

Synthetic data mostly encodes age. Real outage records would give the gas signal its true weight.

3

Time-to-failure

A survival model for "how many months left", not just "fails within 12 months".

4

Inspection outcomes

Learn what inspections actually find, so the value of an inspection is measured, not assumed.

5

Incremental + Postgres

Watermarked loads, SCD Type 2 history, nightly scoring: the scaling path on the previous slide.

6

Monitoring

Drift on inputs, calibration over time, alerts when the model goes stale; retraining plan.

7

UI through the API

One serving path, authentication and roles (engineer approves, planner proposes).

8

Cost of a miss

Use real outage costs per customer type instead of fixed weights; test sensitivity of the plan.

19

Trade-offs: what we chose, what we rejected

Criterion: the simplest design that works, and why

Decision	Chose	Rejected	Why
Pipeline	medallion	one script raw → model	replayable, auditable; quality enforced once
Gold layer	star, conformed dimensions	3NF / snowflake	read-heavy: fewer joins win
Grain	quarterly snapshots	monthly / yearly	tests every 6–12 months: monthly = duplicates, yearly = too few
Refresh	full rebuild (~3 s)	incremental	at this size incremental only adds failure modes
Model	logistic regression	boosting by default	few failures; readable weights; boosting didn't win
Calibration	Platt (sigmoid)	isotonic	isotonic overfits ~40 failures
Inspection value	uncertainty-based	fixed % of risk	inspection ~23× cheaper: a fixed % inspects everything
UI	same functions as API	UI → HTTP → API	one copy of the logic
Packaging	local run	Docker	panel confirmed optional

Thank you: questions?

Leakage-safe by design · honest about uncertainty · every decision explained and approved by a person

```
pip install -r requirements.txt
python -m src.pipeline      # bronze → silver → gold
python -m src.model         # train + evaluate
uvicorn src.api:app         # API   :8000/docs
streamlit run ui/app.py     # UI    :8501
```

Unique0202: architecture, pipeline, features, loader, UI, README

sreerajmahesh: synthetic data, FastAPI service

abinavhub: models, validation, calibration, plan